

Day 35



Fetch API:

What is Fetch API?

Fetch API is a built-in browser feature that lets you request data from servers (APIs). It returns a Promise, so you can use `.then`, `.catch`, or `async/await`.

Basic Syntax

```
fetch("url")  
  
  .then(response => response.json()) // convert to JSON  
  
  .then(data => console.log(data))   // use the data  
  
  .catch(error => console.log(error)); // handle errors
```

Example: a basic example of fetching, using the API.

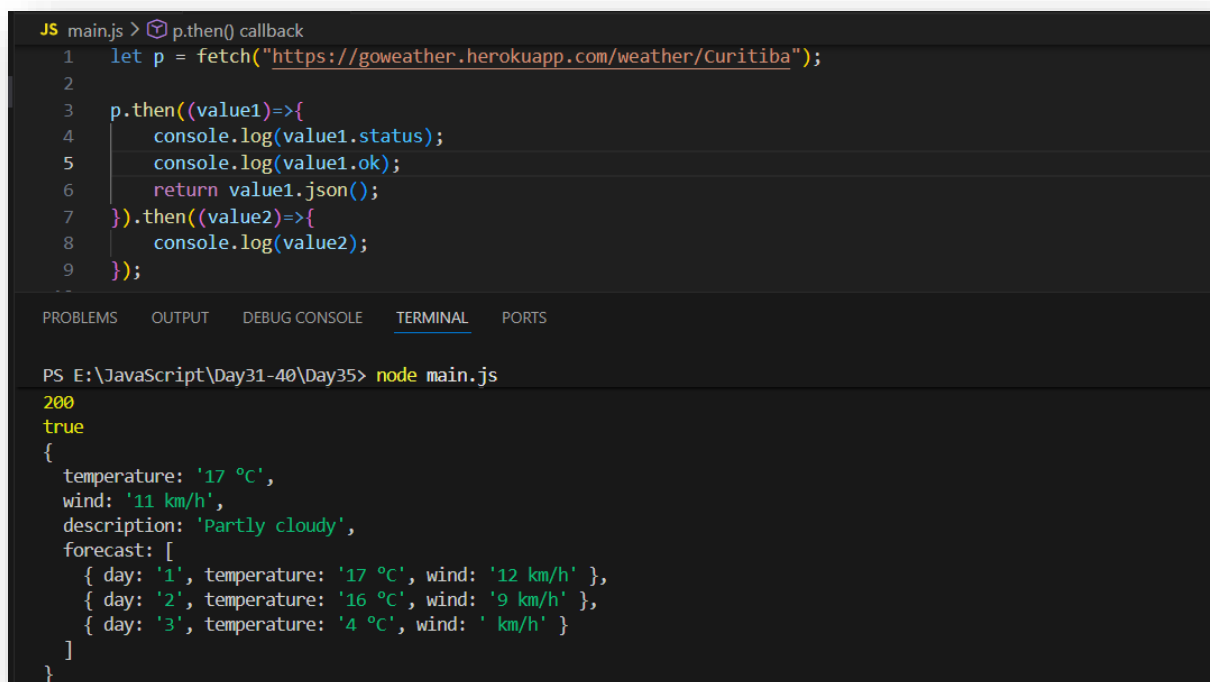
```
JS main.js > ...  
1  let p = fetch("https://goweather.herokuapp.com/weather/Curitiba");  
2  
3  p.then((value1)=>{  
4    return value1.json();  
5  }).then((value2)=>{  
6    console.log(value2);  
7  });  
-  
  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● PS E:\JavaScript\Day31-40\Day35> node main.js  
{  
  temperature: '17 °C',  
  wind: '11 km/h',  
  description: 'Partly cloudy',  
  forecast: [  
    { day: '1', temperature: '17 °C', wind: '12 km/h' },  
    { day: '2', temperature: '16 °C', wind: '9 km/h' },  
    { day: '3', temperature: '4 °C', wind: ' km/h' }  
  ]  
}
```

This code fetches weather data from an API and prints it to the console. The `fetch` function sends a request to the given URL and returns a Promise. In the first `.then()`, the response (`value1`) is converted to JSON and returned. In the second `.then()`, the resulting JSON data (`value2`) is received and logged to the console.

Do you know? Getting a response is a 2 way process?

Getting a response from the Fetch API happens in two steps. First, when you call `fetch()`, the browser connects to the server and gives you a response object, but this object only contains basic information like status code (200, 404), headers, and a placeholder for the body. The actual data (like JSON) is not available yet. Second, you must extract the real data from the response by calling a method such as `response.json()`, `response.text()`, or `response.blob()`. These methods also return a Promise, because converting the body into usable data takes some time. So the fetch process is: (1) fetch returns a response object, (2) you extract the data from the response.

Example: here you can see the status of the response is present



```
JS main.js > p.then() callback
1 let p = fetch("https://goweather.herokuapp.com/weather/Curitiba");
2
3 p.then((value1)=>{
4   console.log(value1.status);
5   console.log(value1.ok);
6   return value1.json();
7 }).then((value2)=>{
8   console.log(value2);
9 });
```

```
PS E:\JavaScript\Day31-40\Day35> node main.js
200
true
{
  temperature: '17 °C',
  wind: '11 km/h',
  description: 'Partly cloudy',
  forecast: [
    { day: '1', temperature: '17 °C', wind: '12 km/h' },
    { day: '2', temperature: '16 °C', wind: '9 km/h' },
    { day: '3', temperature: '4 °C', wind: ' km/h' }
  ]
}
```

This code sends a request to the weather API and logs details about the response before printing the actual data. The fetch call returns a Promise containing the response object. In the first `.then()`, it logs the HTTP status code (`value1.status`) and whether the request was successful (`value1.ok`). Then it converts the response to JSON by returning `value1.json()`. The next `.then()` receives that JSON data (`value2`) and logs the weather information to the console.

Sending POST request with Fetch API:

What is a POST request?

A POST request is used when you want to send data to a server — like form data, login info, or creating a new record.

How do we send POST with Fetch?

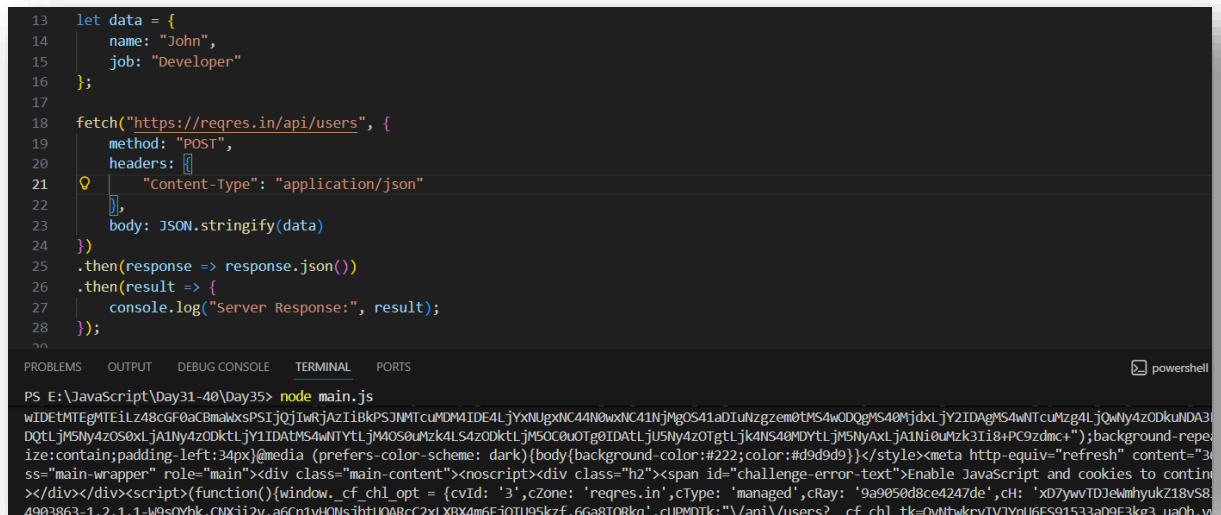
To make a POST request, we add:

- method: "POST"
- headers to tell the server we are sending JSON
- body with the actual data (converted to JSON using `JSON.stringify()`)

Basic Syntax

```
fetch("URL", {  
  
  method: "POST",  
  
  headers: {  
  
    "Content-Type": "application/json"  
  
  },  
  
  body: JSON.stringify(dataToSend)  
  
});
```

Example: a basic example for this



```
13 let data = {  
14   name: "John",  
15   job: "Developer"  
16 };  
17  
18 fetch("https://reqres.in/api/users", {  
19   method: "POST",  
20   headers: {  
21     "Content-Type": "application/json"  
22   },  
23   body: JSON.stringify(data)  
24 })  
25 .then(response => response.json())  
26 .then(result => {  
27   console.log("Server Response:", result);  
28 });
```

The terminal window shows the command `node main.js` being executed in a PowerShell environment. The output is a large, truncated JSON string representing the server response.

This code sends a POST request to an API with a JSON object containing a name and job. The fetch function is given options specifying the HTTP method, headers, and the request body, which is converted to a JSON string. After the request is sent, the first `.then()` converts the server's response into JSON, and the second `.then()` logs that response to the console.

Example: async and await with the fetch API with POST

```
30 async function sendData() {
31   let data = {
32     name: "John",
33     job: "Developer"
34   };
35
36   let response = await fetch("https://reqres.in/api/users", {
37     method: "POST",
38     headers: {
39       "Content-Type": "application/json"
40     },
41     body: JSON.stringify(data)
42   });
43
44   let result = await response.json();
45   console.log("Server Response:", result);
46 }
47
48 sendData();
49
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day31-40\Day35> node main.js

<anonymous_script>:1

<!DOCTYPE html><html lang="en-US"><head><title>Just a moment...</title><meta http-equiv="Content-Type" content="text/html; charset=UTF-8"><meta name="IE=Edge"><meta name="robots" content="noindex,nofollow"><meta name="viewport" content="width=device-width,initial-scale=1"><style>{box-tml{line-height:1.15;-webkit-text-size-adjust:100%;color:#313131;font-family:system-ui,-apple-system,BlinkMacSystemFont,"Segoe UI",Roboto,"h

This version uses async and await to make the POST request easier to read. Inside the sendData function, the fetch call waits for the server response, then await response.json() converts that response to JSON. The final result is printed to the console. Using async/await makes the code look more like normal synchronous code while still handling asynchronous operations.

--The End--